

Compiling the Raytracer to the Native CPU

Generated C, x86-64 assembly, executable, and PPM output

16 August 2026

Abstract

This note adds a native-host backend to the parametric raytracer compiler. A raytracer semantic description is quoted and unquoted, specialized directly into x86-64 AT&T assembly, assembled with ‘as -64’, linked with ‘ld’, and run. The executable produces and validates a 96 by 64 binary PGM image. No C compiler is used for the target build. This backend is intentionally separate from the AArch64 emitter: the high-level ray semantics are shared while the final ABI and machine lowering are target-specific.

1 Compilation pipeline

The tested pipeline is

$$D_{ray} \xrightarrow{\text{quote/unquote}} \text{x86-64 assembly} \xrightarrow{\text{as -64}} \text{object} \xrightarrow{\text{ld}} \text{executable} \rightarrow \text{render.ppm}.$$

The generated C contains a fixed-point orthographic sphere test and a nested pixel loop. It writes an ASCII PPM image with a standard header and one grayscale pixel for each coordinate.

2 Semantic specialization

The description contains width, height, sphere center, radius, and output path. The generator emits these as static constants in the C source. Direct generation and generation through a quoted description are compared byte-for-byte before invoking the host compiler. Therefore the tested path is genuinely staged rather than merely two independently written generators.

3 Native build and execution

The implementation is `cpu_ray_codegen.cpp`. The exact commands used were:

```
g++ -std=c++23 -Wall -Wextra -pedantic -O2 cpu_ray_codegen.cpp -o cpu_ray_codegen
./cpu_ray_codegen > generated_raytracer.s
as --64 generated_raytracer.s -o generated_raytracer.o
ld generated_raytracer.o -o generated_raytracer
./generated_raytracer
```

The resulting executable is a statically linked x86-64 ELF binary on the current machine. Running it succeeds and produces a valid PGM file whose header reports 96×64 pixels and maximum value 255. The output file is non-empty and was identified as Netpbm image data.

4 Relation to the AArch64 backend

The AArch64 backend emits a ray kernel and an explicit pixel sink ABI. The native backend chooses generated C as its portable intermediate representation and delegates final register allocation, calling convention, and object emission to the host C compiler. This is a useful staged bootstrap path: the same semantic description can target both a direct AArch64 emitter and a native CPU toolchain without conflating their ABIs.

5 Limitations

The current renderer uses a binary PGM sink and a single sphere with fixed-point arithmetic. It does not yet implement floating-point vectors, multiple objects, materials, shadows, or acceleration structures. Those extensions can be added to the semantic description while preserving the quote/unquote boundary and the output-image validation step.